

## Methodology Summary

In summary, the implementation is organized around three stages. First, the query gateway converts a query graph into private exact tokens and private semantic bucket tokens. Second, each party evaluates FSS/DPF shares over private HMAC-encoded KG indexes and semantic bucket indexes while preserving structural DFS-based matching. Third, candidate scores are secret-shared, aggregated, ranked through garbled-circuit top- $k$ , and only selected evidence paths are revealed. This staged design keeps the retrieval logic modular while allowing the FSS/DPF, Prio/VDAF, and garbled-circuit components to be integrated independently.

### 0.0.1 Implementation Strategy

The implementation strategy follows the three main stages shown in the methodology figures: query preparation, private multi-party retrieval, and secure aggregation/ranking/reveal. The design is intended for the actual federated KG-RAG setting as follows:

| Component              | Implementation Strategy                                                                                                                        | Planned Details                                                                                                                                                                                              |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Query graph input      | Using the structured query graphs as the retrieval input. Natural-language rewriting can be handled by a trusted or client-side query gateway. | Represent each query as triples with known entities, known relations, and typed unknown variables, e.g. $(h, r, t)$ where $t$ may be UNKNOWN movie.                                                          |
| Query gateway          | The gateway prepares private query material before contacting parties.                                                                         | It normalizes query terms, computes exact HMAC IDs, computes semantic buckets from embeddings, and generates FSS/DPF key shares.                                                                             |
| Text normalization     | Using a shared normalization function across gateway and all parties.                                                                          | Applying case-folding, whitespace trimming, repeated-space collapse, and namespace prefixes such as <code>entity:</code> , <code>relation:</code> , <code>type:</code> , and <code>relation_bucket:</code> . |
| Exact query IDs        | Using HMAC-SHA256 with $K_{\text{setup}}$ for exact identifiers.                                                                               | Computing IDs such as $HMAC(K, \text{entity:tom hanks})$ , $HMAC(K, \text{relation:acted in})$ , and $HMAC(K, \text{type:movie})$ . These IDs are not sent directly to parties.                              |
| Semantic query buckets | Using the embedding model and LSH to preserve semantic retrieval.                                                                              | The gateway embeds query terms such as <code>acted in</code> , maps the vector to one or more LSH buckets, and converts bucket IDs into HMAC bucket tokens.                                                  |
| FSS/DPF query shares   | Generating FSS/DPF shares for both exact IDs and semantic bucket tokens.                                                                       | For each hidden query point $\alpha$ , run $\text{DPF.Gen}(\alpha, \beta = 1)$ and sending one key share to each party.                                                                                      |
| Query privacy boundary | Not transmitting the plaintext query terms, raw embeddings, exact HMAC IDs, or raw bucket IDs to parties.                                      | Parties receive only their own FSS/DPF key shares and protocol metadata such as domain size and query shape.                                                                                                 |

Table 1: Implementation strategy for query preparation.

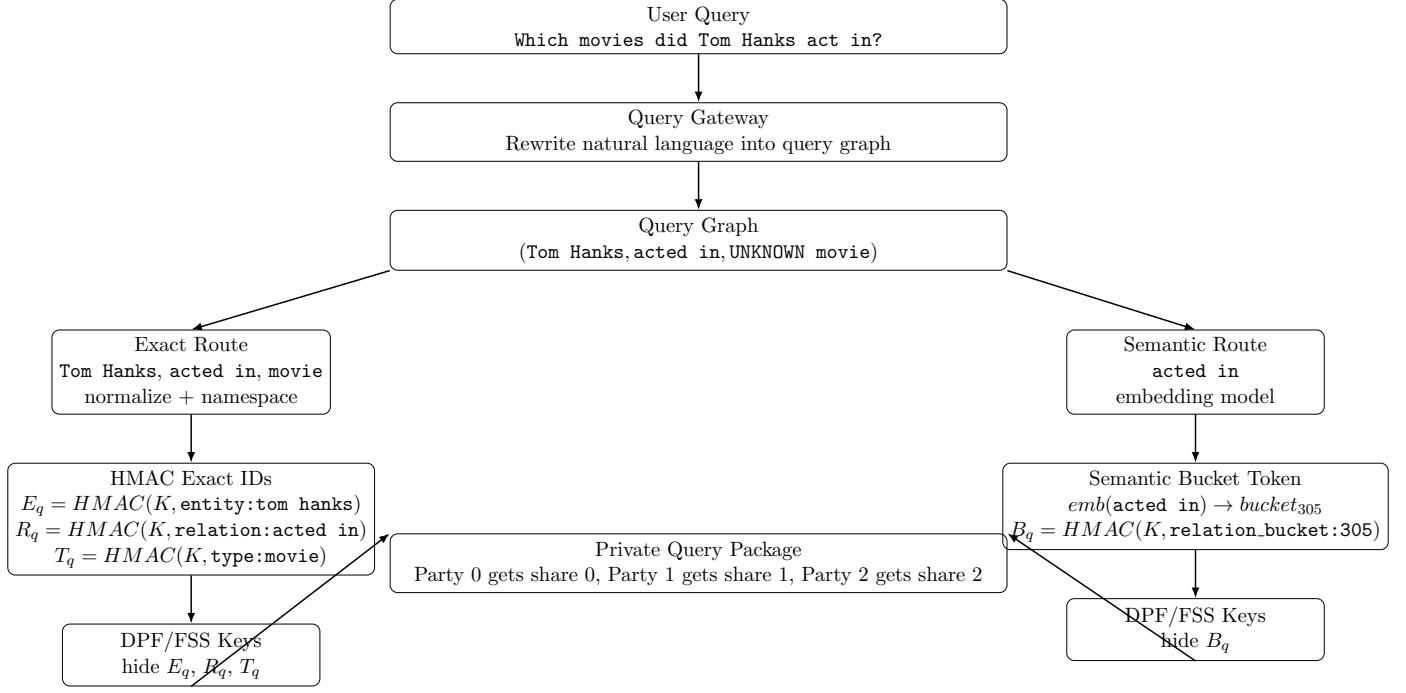
| Component                   | Implementation Strategy                                               | Planned Details                                                                                                                                                                              |
|-----------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Party-local encoding        | Each party independently encodes its own KG and indexes.              | Entities, relations, and types are normalized and converted into HMAC IDs using the shared $K_{\text{setup}}$ . No central vocabulary builder is required.                                   |
| Exact KG index              | Building a local HMAC-encoded adjacency index.                        | Store encoded triples as $H_h \rightarrow H_r \rightarrow [H_t]$ . This supports exact private entity/relation matching and local graph traversal.                                           |
| Type index                  | Building a local HMAC-encoded type index.                             | Store mappings such as $T_{\text{movie}} \rightarrow [H_{\text{ForrestGump}}, H_{\text{CastAway}}]$ for typed unknown-node resolution.                                                       |
| Semantic index              | Building party-local semantic indexes from embeddings.                | Each party embeds local entity/relation labels, maps them to LSH buckets, HMACs the bucket IDs, and stores bucket-token-to-candidate mappings locally.                                       |
| Private exact evaluation    | Evaluating FSS/DPF shares over local exact indexes.                   | Each party computes its share of whether local entity, relation, or type IDs match the hidden query IDs.                                                                                     |
| Private semantic evaluation | Evaluating FSS/DPF shares over local semantic bucket indexes.         | Each party privately checks whether local embedding buckets match the hidden query bucket tokens. This supports semantically related matches such as <b>starred in</b> for <b>acted in</b> . |
| Structural matching         | Preserving the original graph-structural retrieval logic.             | DFS or beam search is performed over encoded handles while enforcing edge direction, relation constraints, type constraints, repeated variables, and multi-hop consistency.                  |
| Cross-party alignment       | Using shared HMAC entity IDs for entities that appear across parties. | If the same entity exists at multiple parties, the same normalized text and $K_{\text{setup}}$ produce the same HMAC ID, enabling encoded cross-party joins.                                 |
| Candidate representation    | Representing matched paths using opaque candidate identifiers.        | Candidate IDs are derived from canonical encoded paths or commitments. Readable evidence paths are not revealed during retrieval.                                                            |
| Candidate scoring           | Computing local candidate scores from private retrieval signals.      | Scores combine exact-match signals, semantic-match signals, type consistency, and structural path confidence. Scores are converted to fixed-point integers.                                  |
| Score sharing               | Secret-share candidate scores and support values.                     | Using additive sharing modulo a large prime: $s = s_0 + s_1 + \dots + s_n p$ . Parties output shares instead of raw scores.                                                                  |

Table 2: Implementation strategy for private multi-party retrieval.

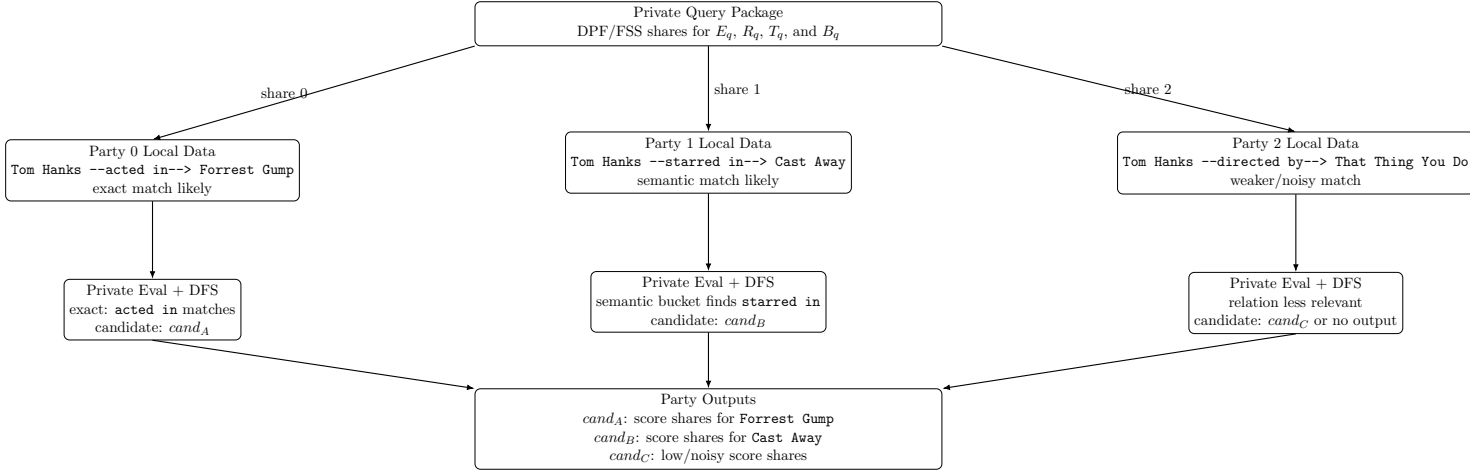
| Component                  | Implementation Strategy                                              | Planned Details                                                                                                                                                                        |
|----------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Secure aggregation         | Aggregating score and support shares by candidate ID.                | The first implementation will use additive-share aggregation. Aggregate scores remain hidden as shares and are not opened for all candidates.                                          |
| Prio/VDAF validation       | Adding validation after the basic retrieval pipeline is functional.  | Using Prio/VDAF-style methods, potentially through <code>libprio-rs</code> , to validate bounded scores, valid support counts, and well-formed measurements.                           |
| Ranking interface          | Defining a stable private ranking interface.                         | Use a <code>SecureTopK</code> abstraction that accepts candidate IDs and secret-shared aggregate scores. This allows mock ranking to be replaced by garbled-circuit ranking later.     |
| Garbled-circuit top- $k$   | Using garbled circuits for private ranking.                          | The circuit computes top- $k$ over hidden aggregate scores. It reveals selected candidate IDs and optionally ranks, but not all scores, score gaps, or full ordering.                  |
| Candidate padding          | Using fixed-size padded candidate arrays for ranking.                | Pad with dummy candidates and low scores before garbled-circuit evaluation to provide a fixed circuit input shape and reduce candidate-count leakage.                                  |
| Controlled evidence reveal | Revealing only selected top- $k$ evidence paths.                     | After ranking, selected candidate IDs are sent to the owning parties. Only those candidates are resolved into readable evidence paths.                                                 |
| Answer generation          | Generating the final answer from selected evidence.                  | The answer generator receives the original question and revealed top- $k$ evidence paths. Private LLM inference can be considered later if the answer model should not see the prompt. |
| Evaluation                 | Comparing secure retrieval against the plaintext federated baseline. | Measuring top- $k$ agreement, answer accuracy, latency, communication size, candidate volume, and privacy-relevant leakage points.                                                     |

Table 3: Implementation strategy for aggregation, private ranking, and evidence reveal.

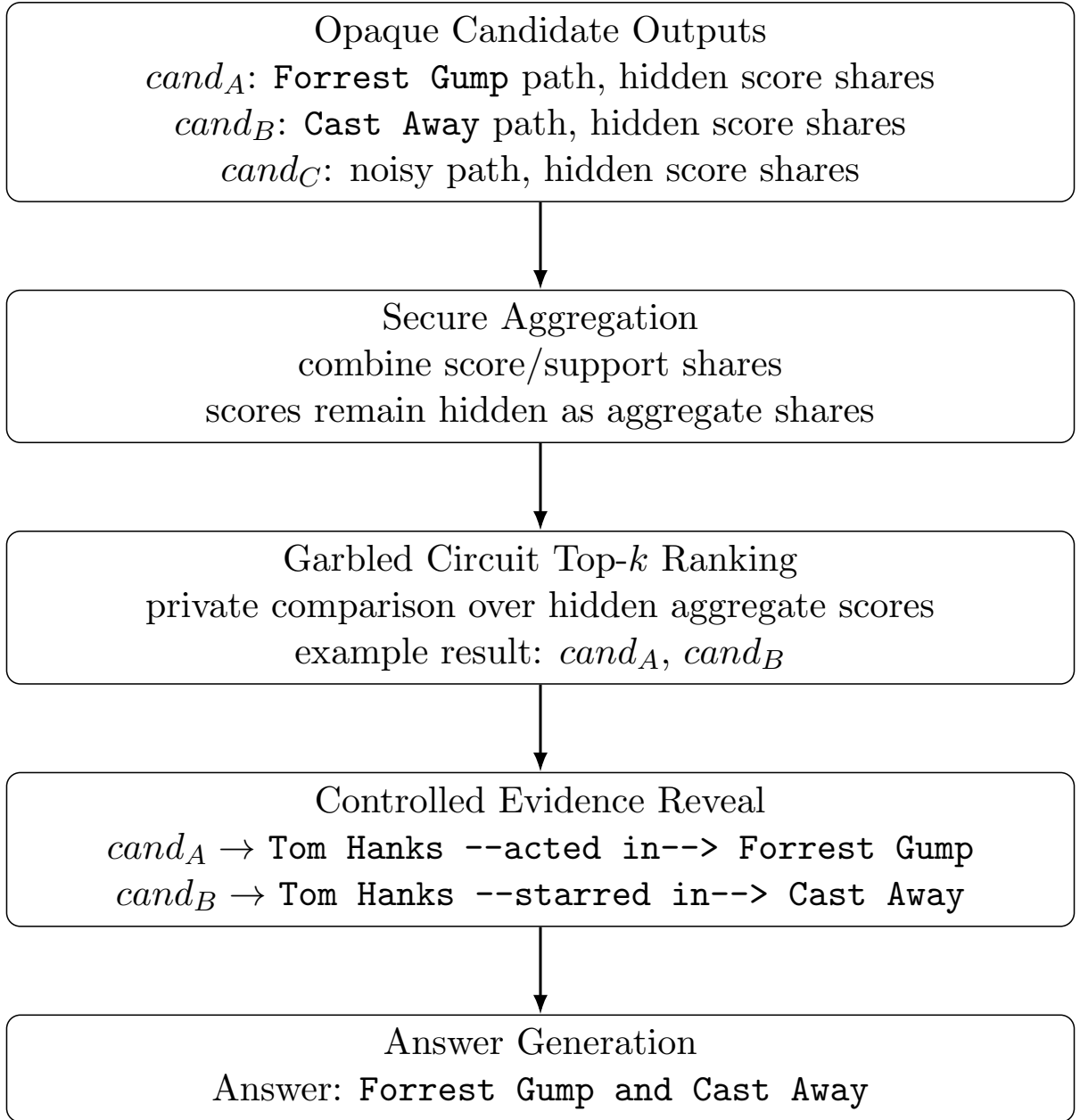
## Methodology diagram



**Phase 1:** Query preparation using the running example. The exact route hides symbolic query terms, while the semantic route hides the embedding-derived bucket for **acted in**.



**Phase 2:** Private retrieval across three parties using the same query. Party 0 contributes an exact candidate, Party 1 contributes a semantic candidate, and Party 2 contributes a weak/noisy candidate or no candidate.



**Phase 3:** Aggregation, private ranking, and final reveal using the same running example. Only selected top-*k* evidence is opened.